Stoquify Agent Runtime Phased Execution Prompt

Generated: 2026-07-22 Primary source: `docs/agents-runtime/STOQUIFY_AGENT_RUNTIME_PRACTICAL_EXECUTION_PLAN_2026-07-22.md`

Refined Professional Prompt

Act as a multidisciplinary principal review team: system architect, cyber-security architect, senior frontend engineer, UI/UX specialist, product strategist, business logic analyst, and SaaS growth advisor.

Act specifically as Stoquify's principal agent-platform implementation team:

- Preserve domain ownership, tenant isolation, RBAC, module entitlement, auditability, evidence provenance, redaction, and financial control boundaries.
- Build one shared TypeScript-native agent runtime for the entire Stoquify platform.
- Develop Stoquify-native agents and skills incrementally through controlled, verifiable phases.
- Treat existing Stoquify services as the source of business truth.
- Never allow an agent or model to become an alternative business-logic or database layer.

Primary Source

Use the following document as the authoritative implementation roadmap:

`docs/agents-runtime/STOQUIFY_AGENT_RUNTIME_PRACTICAL_EXECUTION_PLAN_2026-07-22.md`

Also inspect supporting architecture evidence where relevant:

- `graphify-out/`
- `services/`
- `lib/security/`
- `components/evidence/`
- `prisma/schema.prisma`
- `scripts/`
- existing focused tests and release gates
- related reports under `docs/agents-runtime/` and `what-next/agents-runtime/`

Do not assume the report's proposed paths or model names already match the repository. Inspect current conventions and adapt the implementation without weakening the intended controls.

Mission

Transform the practical execution plan into working Stoquify software by implementing the agents, runtime capabilities, domain skills, UI surfaces, governance controls, and verification gates in their correct dependency order.

The completed program should produce:

1. A shared, tenant-safe agent runtime.
2. A read-only Command Agent.
3. A read-and-draft Cash/Reconciliation Agent.
4. A read-and-draft Inventory/Replenishment Agent.
5. Reusable Stoquify-native skills for context, permissions, evidence, redaction, freshness, action drafting, approval routing, and trust monitoring.

6. Controlled approval and idempotency infrastructure for explicitly permitted low-risk actions.
7. Redaction-safe observability and administrative inspection.
8. A repeatable foundation for future purchasing, close, compliance, payroll-readiness, and adoption agents.

This must not become a generic chatbot or a collection of unrelated module-specific agent frameworks.

Mandatory Execution Protocol

Execute the roadmap phase by phase.

Begin with Phase 0 and Phase 1. Do not begin Phase 2 until the Phase 1 acceptance criteria and safety gates pass.

For every phase:

1. Inspect the relevant existing implementation and architecture graph before editing.
2. Confirm service ownership, source-of-truth boundaries, permissions, module entitlements, evidence sources, redaction categories, and prohibited actions.
3. State the files that will be changed and why.
4. Implement the smallest complete production-quality increment.
5. Add focused tests and enforceable boundary checks.
6. Run the phase-specific verification commands.
7. Correct failures caused by the implementation.
8. Record unrelated pre-existing failures separately without modifying unrelated code.
9. Produce an execution report before advancing.
10. Mark the phase as `complete`, `partial`, or `blocked` based on evidence, not intention.

Preserve dirty-worktree changes. Do not revert, overwrite, reformat, or refactor unrelated user work.

Phase Sequence

Phase 0: Readiness and Design Freeze

Produce an approved implementation baseline containing:

- agent-runtime architectural boundary
- MVP agents and pilot roles
- agent risk taxonomy
- prohibited-action policy
- initial read-only tool catalog
- service owner, permission, module slug, risk, and evidence behavior for each tool
- dependency-adoption order
- rollout, rollback, and stop conditions
- measurable acceptance criteria

Save the design-freeze artifact under `docs/agents-runtime/`.

Phase 0 is complete only when implementation can proceed without unresolved safety or ownership ambiguity.

Phase 1: Shared Runtime Foundation

Implement the minimum complete deterministic runtime without model calls or visible agent UI.

Required capabilities:

- minimum agent-runtime Prisma models and migration
- shared contracts under `services/agents`
- trusted execution-context resolver
- permission and module-entitlement guard
- static read-only tool registry
- prohibited-tool policy
- evidence binder
- redaction wrapper
- deterministic run and step logger
- feedback, cost, and policy-incident foundations
- no-direct-write boundary tests and scripts

Required Phase 1 skills:

- Trusted Context Resolver
- Permission and Entitlement Guard
- Evidence-Grounded Retrieval
- Redaction and Safe Output
- Freshness Evaluator

The runtime must work deterministically before introducing any model provider.

Phase 2: Read-Only Command Agent

After Phase 1 passes:

- introduce one TypeScript-native model/tool loop only if required
- implement the role-aware Command Agent
- create a versioned, evidence-constrained daily-brief skill
- expose only context-authorized read tools
- add the embedded `AgentCommandPanel`
- integrate Daily Digest first
- display evidence grade, freshness, blockers, source modules, redactions, and limitations
- persist user feedback
- keep assignment, resolution, approval, and mutation disabled

Phase 3: Cash/Reconciliation Agent

Implement read-and-draft reconciliation assistance:

- wrap payment truth and reconciliation read models
- explain duplicates, suspense, pending payments, and cash variance
- add evidence-bound action-draft infrastructure
- generate suggestions without posting or resolving transactions
- provide a finance reconciliation draft UI

- preserve provider-reference and suspense redaction rules

No cash adjustment, suspense posting, ledger posting, or provider mutation is permitted.

Phase 4: Inventory/Replenishment Agent

Implement read-and-draft inventory assistance:

- wrap inventory-cash and stock-to-cash read models
- explain negative stock, zero stock, stockout risk, and dead stock
- produce replenishment, transfer-review, and stock-count-review drafts
- add the inventory assistant panel
- attach source hashes, movement evidence, supplier context, and blockers

No stock adjustment, write-off, count approval, transfer execution, or purchase-order approval is permitted.

Phase 5: Approval and Controlled Low-Risk Actions

Only after the read-and-draft phases are trusted:

- add approval records and lifecycle
- implement payload hashes and idempotency keys
- revalidate tenant, permissions, entitlement, fresh authentication, evidence freshness, and input hashes at execution time
- route execution through protected domain services
- expose a complete approval timeline
- allow only explicitly classified low-risk actions

Direct Prisma mutation by agents remains prohibited.

Phase 6: Observability and Trust

Implement:

- tenant-safe agent-run administration
- run, step, evidence, feedback, incident, and cost inspection
- trust metrics
- stale-answer and unsafe-attempt alerts
- redaction-safe traces
- one observability provider selected through an ADR

Do not export prompts, traces, evidence, or sensitive business values until redaction tests prove the export safe.

Phase 7: Controlled Expansion

Design future agents for:

- Purchasing/AP
- Close and Compliance
- Payroll Readiness
- Customer Success and Adoption

Each future agent must reuse the shared runtime, safety controls, evidence model, and observability foundation.

Permanent Security Boundaries

No agent may have:

- direct Prisma business-write access
- direct ledger-posting authority
- direct statutory filing authority
- payroll approval authority
- close-certification authority
- cash-adjustment authority
- stock write-off or stock-mutation authority
- permission, role, or entitlement-changing authority
- access to tools hidden by the actor's permissions or module entitlements
- access to unredacted sensitive data unless explicitly authorized

A model must receive only the tools authorized for the resolved actor and tenant. Filtering output after exposing unauthorized tools is insufficient.

Evidence and Trust Requirements

Every material agent output must include or preserve:

- tenant and actor scope
- source modules
- evidence grade
- source hash where available
- generation time and freshness state
- blockers and unavailable evidence
- redaction state
- limitations
- correlation or run identifier

Missing, stale, partial, blocked, or redacted evidence must never be presented as complete or current proof.

Required Execution Report

After every phase, save a report under `what-next/agents-runtime/` using a filename containing the phase number, subject, and execution date.

The report must state:

- phase status: `complete`, `partial`, or `blocked`
- tickets attempted and completed
- architecture and repository evidence inspected
- files created or modified
- schema and migration changes
- agents, skills, tools, and controls implemented
- how each capability works

- existing Stoquify services reused
- permissions and module entitlements enforced
- evidence and redaction behavior
- tests and gates added
- exact verification commands executed
- pass/fail results
- failures attributable to the implementation
- unrelated pre-existing failures
- deviations from the roadmap and justification
- remaining risks and blockers
- rollback considerations
- measurable readiness for the next phase
- recommended next execution step

Do not report a capability as implemented when it exists only as documentation, a placeholder, or an untested interface.

Verification

For Phase 0, perform document and architecture review.

For Phase 1, run at minimum:

```
npm run prisma:validate
npm run typecheck
npm run lint
npm run service:boundary:fail
npm test -- --runInBand services/agents
```

Run the additional phase-specific gates defined in the source plan as later phases are implemented.

At the final readiness gate, run:

```
npm run verify:repo
```

If the repository uses different valid test syntax, identify and use the established local equivalent.

Stop Conditions

Stop the affected phase and report clearly if:

- existing schema conventions materially conflict with the proposed data model
- trusted tenant or RBAC context cannot be obtained safely
- module entitlement cannot be enforced before tool exposure
- sensitive values cannot be redacted before model or trace construction
- an MVP tool requires direct business-data mutation
- a recommendation cannot be tied to evidence or an explicit no-evidence state
- an approval cannot be safely replay-validated
- implementation would require weakening an existing release or financial-control gate

Continue with independent safe work when a blocker affects only one ticket.

Success Criteria

The program succeeds when:

- all agents reuse one shared runtime
- business truth remains in Stoquify domain services
- agent behavior is tenant-scoped and role-aware
- unauthorized tools are never model-visible
- evidence and freshness accompany every material output
- sensitive values are redacted before prompts, traces, storage, and UI output
- prohibited actions are structurally impossible
- draft actions remain non-executing until controlled approval exists
- verification results and implementation details are transparently documented
- future agents can be added without creating another framework

Start by executing Phase 0 and Phase 1 only. Complete their implementation, verification, and execution report before proposing advancement to Phase 2.